

Role 1: Frontend Development

Domain: Client-Side Architecture, UI/UX Engineering, Data Visualizations & State Management

Core Technology: React 19 • Tailwind CSS v4 • Vite 7 • Radix UI • Recharts • Framer Motion

1. Executive Summary & Scope

The **Frontend Developer** is responsible for the entire user-facing surface of the SatyaCheck platform. The mission of this role is to translate complex, 10-dimensional artificial intelligence forensic fact-checking data into an intuitive, accessible, and editorial visual experience. The user interface eliminates cognitive overload and empowers citizens, researchers, and journalists to verify viral claims in seconds.

Core Engineering Objectives:

- Deliver sub-100ms client route transitions via client-side routing (wouter).
- Provide 4 distinct input verification modes: Raw Text, Web Article URL, Headline, and Screenshot Image.
- Render dynamic, accessible forensic breakdown cards with Recharts visualization graphs.
- Enable private, offline-first client storage via browser `localStorage` with instant CSV export.
- Support accessibility: Web Speech API microphone dictation, keyboard shortcuts (`Ctrl+Enter`), and persistent light/dark themes.

2. Complete Technology Stack

Technology	Version	Specific Role & Implementation Justification
React	19.1.0	Pure JavaScript (ES2024) component-driven reactive user interface.
Vite	7.3.6	Next-generation lightning-fast ES module bundler and hot-module dev server.
Tailwind CSS	4.1.14	Modern design system tokens, responsive utilities, and automated theme styling.
TanStack Query	5.90.21	Server-state management, asynchronous cache invalidation, loading states, and error handling.
Radix UI (shadcn)	Latest	WAI-ARIA compliant accessible headless UI primitives (Dialogs, Tooltips, Tabs, Badges).
Recharts	2.15.4	SVG-based responsive data visualization charts (Forensic confidence pie and metric bars).
Framer Motion	12.4.7	Hardware-accelerated micro-animations, spring-animated metrics, and fluid route transitions.
Vitest	3.0.0	Unit testing framework executing 13 automated tests for UI logic and formatting.

3. Comprehensive Page Catalog

The frontend comprises 11 fully implemented, dedicated routes:

Route	Page Component	Key User Capabilities & Features
/	Home.jsx	Interactive landing portal, live animated counter metrics (Total, Real, Fake checks), feature highlights.
/detect	Detect.jsx	Core verification console: 4 input tabs (Text, URL, Headline, Image), voice dictation, forensic card view.
/chat	Chat.jsx	Real-time streaming AI assistant (Server-Sent Events) with multi-claim analysis and quick prompts.
/dashboard	Dashboard.jsx	Global platform metrics, Recharts distribution graphs, trending misinformation topics.
/history	History.jsx	Personal client-side verification history (up to 100 entries), search, filters, and one-click CSV export.
/forward	Forward.jsx	Dedicated WhatsApp & Telegram chain forward cleaner; strips forwarded noise to analyze manipulation.
/credibility	Credibility.jsx	Domain trust rating lookup; scores media bias (Far Left to Far Right) and journalistic transparency.
/trending	Trending.jsx	Live wire displaying circulating viral conspiracy theories, severity badges, and platform spread.
/quiz	Quiz.jsx	10-question interactive media literacy test with instant educational explanations and score tracking.
/education	Education.jsx	Media Literacy Hub detailing the taxonomy of Misinformation, Disinformation, and Malinformation.
/about	About.jsx	Mission statement, ethical principles, and UN Sustainable Development Goal (SDG 4, 9, 10, 16) alignment.

4. Core Code Implementations

4.1 API Integration Hook (src/hooks/use-analysis.js)

This custom hook utilizes TanStack Query's `useMutation` to dispatch verification requests to the Express backend and automatically invalidate platform statistics caches:

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";

// Dispatches content (text, URL, or image base64) to the backend API
export function useAnalyzeContent() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: async ({ content, type }) => {
      const res = await fetch("/api/analyze", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ content, type }),
      });
      if (!res.ok) {
        const err = await res.json().catch(() => ({}));
        throw new Error(err.message || "Failed to analyze content");
      }
      return res.json();
    },
    onSuccess: () => {
      // Invalidate global stats and history queries so UI re-fetches automatically
      queryClient.invalidateQueries({ queryKey: ["/api/stats"] });
      queryClient.invalidateQueries({ queryKey: ["/api/history"] });
    },
  });
}

// Fetches global platform analytics for the dashboard
export function useStats() {
  return useQuery({
    queryKey: ["/api/stats"],
    queryFn: async () => {
      const res = await fetch("/api/stats");
      if (!res.ok) throw new Error("Failed to load statistics");
      return res.json();
    },
    staleTime: 1000 * 60 * 5, // 5 minutes cache
  });
}
```

4.2 Web Speech API Voice Dictation Hook (src/hooks/use-voice-input.js)

```
import { useState, useCallback } from "react";

export function useVoiceInput(onResult) {
  const [isListening, setIsListening] = useState(false);
  const [error, setError] = useState(null);

  const startListening = useCallback(() => {
    const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
    if (!SpeechRecognition) {
      setError("Speech recognition is not supported in this browser.");
      return;
    }
    const recognition = new SpeechRecognition();
    recognition.continuous = false;
    recognition.interimResults = false;
    recognition.lang = "en-US";

    recognition.onstart = () => { setIsListening(true); setError(null); };
    recognition.onresult = (event) => {
      const transcript = event.results[0][0].transcript;
      onResult(transcript);
    };
    recognition.onerror = (e) => { setError(e.error); setIsListening(false); };
    recognition.onend = () => setIsListening(false);

    recognition.start();
  }, [onResult]);

  return { isListening, error, startListening };
}
```

4.3 Client-Side Storage & CSV Export (src/pages/History.jsx)

```
const STORAGE_KEY = "satyacheck_history";

export function saveToHistory(entry) {
  try {
    const existing = JSON.parse(localStorage.getItem(STORAGE_KEY) || "[]");
    const newEntry = {
      ...entry,
      id: Date.now().toString(),
      timestamp: new Date().toISOString(),
    };
    const updated = [newEntry, ...existing].slice(0, 100); // Cap at 100 entries
    localStorage.setItem(STORAGE_KEY, JSON.stringify(updated));
    return updated;
  } catch {
    return [];
  }
}

export function exportHistoryToCSV(entries) {
  const headers = ["Date", "Type", "Content", "Verdict", "Confidence"];
  const rows = entries.map(e => [
    e.timestamp,
    e.type,
    `${(e.content || "").replace(/"/g, '"')}`,
    e.verdict,
    `${e.confidence}%`
  ]);
  const csvContent = "data:text/csv;charset=utf-8," + [headers, ...rows].map(r =>
r.join(",")).join("\n");
  const encodedUri = encodeURI(csvContent);
  const link = document.createElement("a");
  link.setAttribute("href", encodedUri);
  link.setAttribute("download", `satyacheck_history_${new Date().toISOString().slice(0,10)}.csv`);
  document.body.appendChild(link);
  link.click();
  document.body.removeChild(link);
}
```

5. UI/UX & Design System Standards

- **Color Architecture:** Emerald Green primary (#059669) symbolizing Truth (*Satya*), Slate Dark Neutral (#0f172a), Slate Light (#f8fafc), Alert Red (#dc2626) for Fake news, Amber (#d97706) for Misleading.
- **Theme Persistence:** Implemented via `localStorage.getItem("satya-theme")`; automatically prevents flash of wrong theme on page load.
- **Typography:** Proportional font scaling utilizing system sans-serif hierarchy with high editorial contrast.
- **Feedback Mechanisms:** Interactive confetti explosion via `canvas-confetti` on VERIFIED REAL results to reward factual media consumption.

6. Unit Testing & Quality Assurance

The frontend features a dedicated Vitest suite (**13 automated test cases** passing in ~200ms):

- `utils.test.js`: Tests Tailwind class conflict resolution (e.g. `px-2 py-1` overridden by `px-4`) and conditional class truthiness.
- `verdict.test.js`: Verifies verdict badges, manipulation index thresholds (>60 High, >35 Moderate, <35 Low), and Recharts pie data summing to 100%.
- `storage.test.js`: Validates `localStorage` item bounding (maximum 100 entries) and CSV column escaping.

7. Viva & Technical Defense Q&A

Q1: Why did you choose React 19 with Vite instead of Next.js?

A: SatyaCheck is designed as an ultra-fast client-side single page application (SPA). Vite 7 provides instant hot-module replacement (sub-50ms) and outputs lightweight static bundles that can be deployed to any CDN or static server without server-rendering overhead.

Q2: How does the client prevent UI freezing during long AI inferences?

A: All network calls are managed asynchronously using TanStack React Query. The UI displays responsive skeleton loaders and animated progress bars, ensuring the browser main thread remains completely interactive.

Q3: How is user privacy guaranteed on the frontend?

A: History records are stored exclusively in the user's browser `localStorage`. No search history or tracking cookies are sent to third-party ad networks.